

E-COMMERCE MARKETPLACE

Backend Schema Reference

*Complete Mongoose Schema Definitions with Field Specifications,
Indexes, Middleware & Business Rule Annotations*

Version	1.0
Date	June 22, 2026
Status	Draft
Database	MongoDB (Mongoose ODM)
Runtime	Node.js · Express.js
Collections	10 (User, Product, Cart, Wishlist, Order, Review, Coupon, Notification, SupportTicket, ChatHistory)
Classification	Confidential

This document is the authoritative schema reference for all MongoDB collections in the platform. It derives from the PRD, TRD, and project requirements, and is aligned with the UI/UX Brief.

Table of Contents

1.	Overview & Architecture	3
2.	User	4
3.	Product	6
4.	Cart	8
5.	Wishlist	9
6.	Order	10
7.	Review	13
8.	Coupon	14
9.	Notification	15
10.	SupportTicket	16
11.	ChatHistory	18
12.	Index Export (index.js)	19
13.	Cross-Collection Relationships	20
14.	Business Rules Summary	21

1. Overview & Architecture

The platform uses a **monolithic MVC architecture** on Express.js with MongoDB as the sole data store, accessed via the Mongoose ODM. All schemas follow a consistent set of conventions described below.

Global Conventions

- **timestamps: true** — Every schema automatically gets createdAt and updatedAt fields via Mongoose.
- **ObjectId refs** — All cross-collection references use Types.ObjectId with a ref string for .populate().
- **Snapshots** — Order items and shipping address are copied (snapshotted) at purchase time — changes to the source document do not affect historical orders.
- **Soft deletes** — Products use a status field (unpublished) rather than hard deletion. User deletion is guarded by business rules.
- **Enums** — String enum fields are validated at the schema level; constants are also exported for use in service/controller layers.
- **Indexes** — All query-critical fields are indexed. Compound indexes cover the most common filter+sort combinations.
- **TTL indexes** — Notification (90 days) and ChatHistory (180 days) use TTL indexes for automatic document expiry.
- **Pre/Post hooks** — Middleware is used for auto-status transitions, slug generation, status history logging, and denormalised stat recalculation.

Collection Summary

Model	Collection	Purpose
User	users	Customers, sellers, and admins in one collection — role field discriminates.
Product	products	Catalogue listings with inventory, pricing, media, and full-text search index.
Cart	carts	One document per customer; ephemeral — validated again at checkout.
Wishlist	wishlists	Saved product references per customer for deferred purchase intent.
Order	orders	Immutable purchase record with snapshots, lifecycle statuses, and return/refund tracking.
Review	reviews	Verified-purchase reviews; post-save hook keeps Product rating stats in sync.
Coupon	coupons	Discount codes with type/value/constraints; calculateDiscount() method.
Notification	notifications	In-app alerts with polymorphic ref; TTL auto-expiry after 90 days.
SupportTicket	supporttickets	Complaint/support records with escalation and full status history.
ChatHistory	chathistories	AI chatbot session logs per user; TTL expiry after 180 days.

2. User

User.js

Stores all platform users (customers, sellers, admins) in a single collection discriminated by role.

2.1 AddressSchema (Sub-document)

Field	Type	Req	Description / Notes
label	String	No	Human-readable label e.g. "Home", "Work". Default: "Home".
fullName	String	Yes	Recipient full name for delivery.
phone	String	Yes	Contact phone for courier.
line1	String	Yes	Street address line 1.
line2	String	No	Apartment/suite/floor (optional).
city	String	Yes	City name.
state	String	Yes	State or province.
postalCode	String	Yes	ZIP / PIN code.
country	String	Yes	Default: "India".
isDefault	Boolean	No	Whether this is the user's default shipping address.

2.2 UserSchema (Main)

Field	Type	Req	Description / Notes
name	String	Yes	Full display name. Trimmed.
email	String	Yes	Unique, lowercased. Used for login and notifications.
passwordHash	String	Yes	bcrypt hash (rounds=12). Never returned in API responses.
role	String	Yes	Enum: customer seller admin. Default: customer.
status	String	Yes	Enum: active suspended banned. Default: active.
isVerified	Boolean	No	Email verification flag. Default: false.
verifyToken	String	No	Short-lived token for email verification flow.
resetToken	String	No	Short-lived token for password reset.
resetTokenExpiry	Date	No	Expiry timestamp for resetToken.

Field	Type	Req	Description / Notes
avatar	String	No	Cloudinary URL for profile picture.
phone	String	No	Optional contact phone number.
addresses	[Address]	No	Embedded array of address sub-documents.
sellerStatus	String	No	Enum: pending approved rejected null. Null for customers.
businessName	String	No	Seller only. Trade/brand name.
businessType	String	No	Seller only. E.g. "Sole Proprietor", "Pvt Ltd".
taxId	String	No	Seller only. GST / Tax registration number.
bankDetails	Object	No	Seller only. Nested: accountHolder, accountNumber, ifscCode, bankName.
sellerRejectionNote	String	No	Admin note sent when seller application is rejected.

2.3 Indexes

Index Field(s)	Type	Purpose
email	Unique	Primary login lookup; unique constraint.
role, status	Compound	Admin user management: filter by role and account status.
sellerStatus	Single	Admin seller approval queue: filter pending/approved/rejected.

2.4 Virtuals

- isSeller** — returns true when role === "seller" AND sellerStatus === "approved".

2.5 Business Rules Enforced Here

- BR-03 Account Deletion Guard** — enforced in AuthService before calling User.deleteOne(). Checks zero pending orders, open refunds, active complaints, and outstanding payments.

3. Product

Product.js

Product catalogue. Includes full-text search index, auto-slug generation, and stock-status middleware.

3.1 SpecificationSchema (Sub-document)

Field	Type	Req	Description / Notes
key	String	Yes	Spec name, e.g. "Brand", "Colour", "Weight".
value	String	Yes	Spec value, e.g. "Sony", "Black", "1.2 kg".

3.2 ProductSchema (Main)

Field	Type	Req	Description / Notes
seller	ObjectId	Yes	Ref: User. The seller who owns this listing.
title	String	Yes	Product title. Trimmed. Included in full-text index.
slug	String	Yes	URL-safe unique identifier. Auto-generated from title + timestamp if not provided.
description	String	No	Long-form description. Included in full-text index.
category	String	Yes	Top-level category string. Used for filtering.
images	[String]	No	Cloudinary URLs. images[0] is the cover/thumbnail.
price	Number	Yes	Current selling price. Min: 0.
comparePrice	Number	No	Original/strikethrough price for showing discounts.
sku	String	No	Stock Keeping Unit. Unique sparse index. Trimmed.
stock	Number	Yes	Current inventory count. Min: 0. Default: 0.
lowStockThreshold	Number	No	Triggers low-stock alert when stock falls below this. Default: 5.
status	String	Yes	Enum: active out_of_stock unpublished flagged pending_review.
specifications	[Spec]	No	Array of key-value specification sub-documents.
averageRating	Number	No	Denormalised. Recalculated by Review post-save hook. Range 0–5.
reviewCount	Number	No	Denormalised. Recalculated by Review post-save hook.
adminNote	String	No	Internal note added during product moderation.
isFeatured	Boolean	No	Marks product for homepage featured section. Default: false.

Field	Type	Req	Description / Notes
tags	[String]	No	Searchable tag array. Included in full-text index. Lowercased.

3.3 Indexes

Index Field(s)	Type	Purpose
seller	Single	Seller portal: list own products.
category, status	Compound	Storefront category browse + status filter.
slug	Unique	URL routing — product detail page lookup.
sku	Unique Sparse	Product lookup by SKU; sparse allows null.
status, averageRating (desc)	Compound	Default sort: active products by rating.
title, description, tags	Text	Full-text keyword search across all three fields.
isFeatured, status	Compound	Homepage featured section query.

3.4 Middleware (pre-save hooks)

- **Stock → Status sync** — when stock is modified to 0 and status is "active", status is automatically set to "out_of_stock". When stock rises above 0 from "out_of_stock", status reverts to "active". Implements BR-04.
- **Slug generation** — if slug is absent on pre-validate, one is derived from title (lowercased, hyphenated) with a Date.now() suffix to guarantee uniqueness.

4. Cart

Cart.js

One document per customer. Ephemeral — price and stock are re-validated at checkout.

4.1 CartItemSchema (Sub-document)

Field	Type	Req	Description / Notes
product	ObjectId	Yes	Ref: Product. The item being held in cart.
quantity	Number	Yes	Units in cart. Min: 1.
priceAtAdd	Number	Yes	Snapshot of product.price when item was added. Used for cart total display only — re-validated at checkout.

4.2 CartSchema (Main)

Field	Type	Req	Description / Notes
customer	ObjectId	Yes	Ref: User. Unique — one cart document per customer.
items	[CartItem]	No	Embedded array of cart item sub-documents.
couponCode	String	No	Temporarily stored coupon code (uppercased). Re-validated at checkout.
discount	Number	No	Calculated discount amount. Default: 0.

4.3 Indexes

Index Field(s)	Type	Purpose
customer	Unique	One cart per customer; fast lookup by user.
items.product	Single	Check if specific product is already in any cart (admin use / cleanup).

4.4 Virtuals

- subtotal** — sum of (priceAtAdd × quantity) across all items.
- total** — subtotal minus discount (floored at 0).
- itemCount** — total unit count across all items.

4.5 Business Rules Enforced (Service Layer)

- **BR-01 Cart Stock Guard** — `CartService.addItem()` must query `Product.stock > 0` before pushing to `items[]`. Schema alone does not enforce this.

5. Wishlist

Wishlist.js

Saved product references per customer. One document per user.

5.1 WishlistSchema (Main)

Field	Type	Req	Description / Notes
customer	ObjectId	Yes	Ref: User. Unique — one wishlist per customer.
products[].product	ObjectId	Yes	Ref: Product. The saved product.
products[].addedAt	Date	No	Timestamp when product was added to wishlist. Default: now.

5.2 Indexes

Index Field(s)	Type	Purpose
customer	Unique	One wishlist document per customer.
products.product	Single	Check if a specific product is wishlisted by any user.

5.3 Service-Layer Note

Use MongoDB's `$addToSet` operator when adding a product to prevent duplicate entries. `WishlistService` should also expose a `toggleWishlist()` method that adds or removes based on current state.

6. Order

Order.js

Immutable purchase record. Snapshots pricing and address at checkout. Full lifecycle and return/refund tracking.

6.1 AddressSnapshotSchema (Sub-document)

A plain copy of the customer's delivery address at time of checkout. Not a reference — changes to the User's address do not affect this record.

Field	Type	Req	Description / Notes
fullName	String	No	Snapshot of recipient name.
phone	String	No	Snapshot of contact phone.
line1	String	No	Street address line 1.
line2	String	No	Street address line 2 (optional).
city	String	No	City at time of order.
state	String	No	State at time of order.
postalCode	String	No	PIN / ZIP at time of order.
country	String	No	Country at time of order.

6.2 OrderItemSchema (Sub-document)

Field	Type	Req	Description / Notes
product	ObjectId	No	Ref: Product. Kept for linking; may be null if product deleted.
seller	ObjectId	No	Ref: User (seller). For seller order routing.
title	String	Yes	Product title snapshot at time of purchase.
image	String	No	Cover image URL snapshot.
price	Number	Yes	Unit price snapshot at time of purchase.
quantity	Number	Yes	Units purchased. Min: 1.
sku	String	No	SKU snapshot at time of purchase.

6.3 StatusHistorySchema (Sub-document)

Field	Type	Req	Description / Notes
status	String	Yes	The status value at this point in history.
note	String	No	Optional human-readable note (e.g. tracking number).
changedBy	ObjectId	No	Ref: User. Seller or admin who triggered the change.
timestamp	Date	No	When the change occurred. Default: now.

6.4 OrderSchema (Main)

Field	Type	Req	Description / Notes
customer	ObjectId	Yes	Ref: User. The purchasing customer.
items	[OrderItem]	Yes	Array of order item sub-documents (snapshots).
shippingAddress	AddressSnapshot	Yes	Snapshot of delivery address at checkout.
subtotal	Number	Yes	Sum of (price × quantity) across all items.
discount	Number	No	Coupon discount applied. Default: 0.
tax	Number	No	Tax amount. Default: 0.
total	Number	Yes	Final amount charged = subtotal - discount + tax.
couponCode	String	No	Applied coupon code (uppercased).
paymentMethod	String	Yes	Enum: card upi cod.
paymentStatus	String	Yes	Enum: pending paid refunded failed. Default: pending.
paymentRef	String	No	Gateway transaction / reference ID.
status	String	Yes	Order lifecycle status (see 6.5). Default: placed.
statusHistory	[StatusHistory]	No	Auto-appended on every status change by pre-save hook.
returnReason	String	No	Customer-provided reason for return request.
returnImages	[String]	No	Cloudinary URLs of product photos submitted with return.
returnRequestedAt	Date	No	Timestamp of return request submission.
refundAmount	Number	No	Confirmed refund amount (set by seller/admin on approval).
estimatedDelivery	Date	No	Estimated delivery date set when order is shipped.

6.5 Order Status Lifecycle

Status	Meaning
--------	---------

placed	Order received; stock decremented; confirmation email sent.
processing	Seller has acknowledged and is preparing the order.
shipped	Order dispatched; tracking info may be added to statusHistory note.
out_for_delivery	With local courier for last-mile delivery.
delivered	Confirmed delivered. Return/Review CTAs become available.
cancelled	Order cancelled by customer or seller before shipping.
return_requested	Customer submitted a return request with reason + photos.
return_accepted	Seller/admin accepted the return request.
return_rejected	Seller/admin rejected the return request.
returned	Physical goods received back by seller.
refund_initiated	Refund process started with payment gateway.
refunded	Refund completed; paymentStatus set to refunded.

6.6 Indexes

Index Field(s)	Type	Purpose
customer, createdAt (desc)	Compound	Customer order history, newest first.
items.seller, status	Compound	Seller portal: orders containing seller's products filtered by status.
status	Single	Admin and analytics: filter all orders by status.
paymentStatus	Single	Finance reconciliation: filter by payment state.
createdAt (desc)	Single	Platform-wide analytics: orders by date.

6.7 Middleware

- pre('save')** — whenever status is modified, a new StatusHistory entry { status, timestamp } is automatically pushed. Controllers do not need to manage history manually.

7. Review

Review.js

Verified-purchase product reviews. Post-save hook keeps Product rating stats denormalised.

7.1 ReviewSchema (Main)

Field	Type	Req	Description / Notes
product	ObjectId	Yes	Ref: Product. The product being reviewed.
customer	ObjectId	Yes	Ref: User. The reviewer.
order	ObjectId	Yes	Ref: Order. Ensures the reviewer actually purchased the product (verified-purchase gate).
rating	Number	Yes	Star rating 1–5 (integer).
title	String	No	Short review headline. Max 120 characters.
body	String	No	Full review text. Max 2000 characters.
images	[String]	No	Cloudinary URLs of review photos (optional).
status	String	Yes	Enum: published flagged removed. Default: published.
adminNote	String	No	Moderation note added by admin when flagging/removing.
helpfulCount	Number	No	Upvotes from other customers. Default: 0.

7.2 Indexes

Index Field(s)	Type	Purpose
product, status, createdAt (desc)	Compound	Product detail page: published reviews for a product, newest first.
customer	Single	Customer account: list own reviews.
product, customer, order	Unique Compound	Enforce one review per customer per order-item.

7.3 Post-Save Hook

After every save, an aggregation pipeline recalculates **averageRating** and **reviewCount** on the parent Product document (published reviews only). This keeps the product listing stats accurate without requiring a separate cron job.

8. Coupon

Coupon.js

Discount codes with type/value/constraints. Includes `isUsable` virtual and `calculateDiscount()` method.

8.1 CouponSchema (Main)

Field	Type	Req	Description / Notes
code	String	Yes	Unique discount code (uppercased). E.g. "SAVE20".
type	String	Yes	Enum: percent flat.
value	Number	Yes	Discount value. Percentage (0–100) for percent type; absolute amount for flat type.
maxDiscount	Number	No	Cap on discount for percent coupons. E.g. max █500 off even if % is higher.
minOrderValue	Number	No	Minimum cart total required to apply coupon. Default: 0.
expiresAt	Date	Yes	Coupon is invalid after this date.
usageLimit	Number	No	Maximum total redemptions allowed. Null = unlimited.
usageCount	Number	No	Number of times this coupon has been redeemed. Default: 0.
isActive	Boolean	No	Admin toggle. Inactive coupons are rejected at checkout. Default: true.
createdBy	ObjectId	No	Ref: User (admin). Who created this coupon.
description	String	No	Human-readable description shown to customers.

8.2 Indexes

Index Field(s)	Type	Purpose
code	Unique	Primary lookup at checkout; enforces uniqueness.
isActive, expiresAt	Compound	Admin coupon list filtered by active/expired state.

8.3 Virtuals & Methods

- isExpired** (virtual) — returns true when current date > expiresAt.
- isUsable** (virtual) — true when isActive AND NOT isExpired AND (usageLimit is null OR usageCount < usageLimit).
- calculateDiscount(orderTotal)** (instance method) — returns discount amount for a given order total. Returns 0 if not usable or order total is below minOrderValue. Handles both percent (with maxDiscount cap) and flat types.

9. Notification

Notification.js

In-app alerts with polymorphic document ref. TTL index auto-deletes after 90 days.

9.1 NotificationSchema (Main)

Field	Type	Req	Description / Notes
recipient	ObjectId	Yes	Ref: User. Who receives this notification.
type	String	Yes	Enum — see type list below.
title	String	Yes	Short notification headline (e.g. "Your order has shipped").
message	String	Yes	Full notification body text.
isRead	Boolean	No	Read state. Default: false. Set true when user views.
relatedModel	String	No	Polymorphic ref type: Order Product SupportTicket Review null.
relatedId	ObjectId	No	ObjectId of the related document (used with relatedModel).
actionUrl	String	No	Deep link path within the app (e.g. "/orders/abc123").

9.2 Notification Types

[order_placed](#) · [order_status_update](#) · [return_status_update](#)

[refund_initiated](#) · [refund_completed](#) · [complaint_update](#)

[seller_approved](#) · [seller_rejected](#) · [low_stock_alert](#)

[review_flagged](#) · [coupon_created](#) · [general](#)

9.3 Indexes

Index Field(s)	Type	Purpose
recipient, isRead, createdAt (desc)	Compound	In-app notification panel: unread first, paginated.
createdAt	TTL (90d)	Auto-deletes notifications older than 90 days.

10. SupportTicket

SupportTicket.js

Complaints and support requests. Full status lifecycle, escalation, assignment, and status history.

10.1 TicketStatusHistorySchema (Sub-document)

Field	Type	Req	Description / Notes
status	String	Yes	Status value at this point in history.
note	String	No	Admin note accompanying this status change.
changedBy	ObjectId	No	Ref: User (admin). Who made the change.
timestamp	Date	No	When the change occurred. Default: now.

10.2 SupportTicketSchema (Main)

Field	Type	Req	Description / Notes
customer	ObjectId	Yes	Ref: User. The customer who raised the ticket.
order	ObjectId	No	Ref: Order. Optional link to the relevant order.
category	String	Yes	Enum — see categories below.
subject	String	Yes	Brief issue summary. Max 200 characters.
description	String	Yes	Detailed description. Max 3000 characters.
attachments	[String]	No	Cloudinary URLs of attached images (max 3).
status	String	Yes	Enum: open in_review escalated resolved closed. Default: open.
statusHistory	[StatusHistory]	No	Auto-appended on every status change by pre-save hook.
assignedTo	ObjectId	No	Ref: User (admin). Admin handling this ticket.
adminNotes	String	No	Internal resolution notes (not shown to customer by default).
resolvedAt	Date	No	Auto-set when status changes to resolved.
closedAt	Date	No	Auto-set when status changes to closed.
isEscalated	Boolean	No	Quick filter flag. Default: false.
escalationNote	String	No	Reason for escalation.

10.3 Ticket Categories

order_issue · payment_issue · return_refund · product_quality · seller_conduct · account_issue · other

10.4 Indexes

Index Field(s)	Type	Purpose
customer, createdAt (desc)	Compound	Customer account: own tickets newest first.
status, createdAt (desc)	Compound	Admin queue: filter by status + date sort.
isEscalated, status	Compound	Admin escalation queue view.
assignedTo	Single	Admin workload: tickets assigned to a specific admin.

10.5 Middleware

- pre('save')** — on status change, appends entry to statusHistory. Also auto-sets resolvedAt when status → resolved, and closedAt when status → closed.

11. ChatHistory

ChatHistory.js

AI chatbot session logs. TTL auto-expires sessions after 180 days. Escalation link to SupportTicket.

11.1 ChatMessageSchema (Sub-document)

Field	Type	Req	Description / Notes
role	String	Yes	Enum: user assistant.
content	String	Yes	Message text. Max 4000 characters.
timestamp	Date	No	When message was sent. Default: now.
context.type	String	No	Optional context type: product order null.
context.refId	ObjectId	No	ObjectId of the referenced product or order.

11.2 ChatHistorySchema (Main)

Field	Type	Req	Description / Notes
user	ObjectId	Yes	Ref: User. Session owner.
sessionId	String	Yes	UUID generated client-side per chat session. Unique.
messages	[ChatMessage]	No	Array of message sub-documents in chronological order.
isActive	Boolean	No	true while the session is open. false when closed. Default: true.
endedAt	Date	No	Timestamp when session was closed.
escalated	Boolean	No	true if chatbot escalated to a support ticket. Default: false.
supportTicketId	ObjectId	No	Ref: SupportTicket. Populated when escalated = true.

11.3 Indexes

Index Field(s)	Type	Purpose
user, createdAt (desc)	Compound	User chat history list, newest first.
sessionId	Unique	Session resume / continuation lookup.
createdAt	TTL (180d)	Auto-deletes sessions older than 180 days.

12. Index Export (index.js)

A barrel file that imports and re-exports all 10 Mongoose models. Import models anywhere in the codebase using a single require path:

```
// Single import - all models
const { User, Product, Order } = require('./models');

// Or import the full namespace
const models = require('./models');

// Available exports:
// User, Product, Cart, Wishlist, Order,
// Review, Coupon, Notification, SupportTicket, ChatHistory
```

13. Cross-Collection Relationships

The diagram below summarises how the 10 collections reference each other. All references are ObjectId fields with a ref string enabling Mongoose .populate().

Relationship	Field	Notes
User ← Product	seller	Every Product belongs to one approved Seller (User).
User ← Cart	customer	One Cart per User. Unique index enforces 1:1.
User ← Wishlist	customer	One Wishlist per User. Unique index enforces 1:1.
User ← Order	customer	One User can have many Orders.
User ← Review	customer	Reviewer identity and verified-purchase link.
User ← Notification	recipient	Notifications are addressed to a specific User.
User ← SupportTicket	customer	Complaint raised by a User.
User ← ChatHistory	user	Chat session owned by a User.
Product ← Cart.items	product	Cart items hold a ref to the Product being held.
Product ← Wishlist	products[].product	Wishlist stores refs to saved Products.
Product ← Order.items	product	Order items hold a snapshot ref (may be null if deleted).
Product ← Review	product	Review is written for a specific Product.
Order ← Review	order	Verified-purchase gate: review requires an Order ref.
Order ← SupportTicket	order	Optional link to the associated Order.
SupportTicket ← ChatHistory	supportTicketId	When chatbot escalates, session links to ticket.
User (admin) ← Coupon	createdBy	Tracks which admin created each coupon.

14. Business Rules Summary

The following business rules are defined in the PRD and TRD. This table maps each rule to where it is enforced in the schema layer vs. the service/controller layer.

ID	Rule	Description	Service Layer	Schema Layer
BR-01	Cart Stock Guard	A product with stock = 0 cannot be added to any cart. UI disables Add to Cart and shows "Out of Stock".	CartService.addItem() — queries Product.stock > 0 before pushing.	Product pre-save sets status → out_of_stock when stock hits 0.
BR-02	Seller Approval Gate	Seller listings are only visible on the storefront after admin approval.	ProductController — only queries status = "active" products; sellers with sellerStatus != "approved" cannot create active listings.	User.sellerStatus enum validated at schema level.
BR-03	Account Deletion Guard	An account can only be deleted when there are zero pending orders, open refunds, active complaints, payments, or obligations.	AuthService.deleteAccount() — queries Order, SupportTicket before calling User.deleteOne().	No schema-level hook (requires cross-collection query).
BR-04	Auto Out-of-Stock	When stock is decremented to 0 on order placement, product status automatically becomes out_of_stock.	OrderService.checkout() — decrements Product.stock; Product pre-save hook handles status transition.	Product pre-save middleware: stock → 0 triggers status = "out_of_stock".
BR-05	Coupon Constraints	Coupons enforce minimum order value, expiry date, and usage limits. Invalid coupons are rejected at checkout.	OrderService.checkout() — calls coupon.isUsable and coupon.calculateDiscount() before applying.	Coupon.isUsable virtual; Coupon.isExpired virtual; Coupon.calculateDiscount() method.

End of Backend Schema Reference · E-Commerce Marketplace · v1.0 · June 2026 · Confidential